

1. INTRODUCTION

The demand of static random access memory cell in embedded memories is rapidly increasing. To design SRAM is continuously becoming difficult due to limitation of weaker write margin, degraded data stability and higher leakage power consumption. In order to overcome these limitations, new 8T SRAM has been explored. The simulation results depicts better data stability, low access delay and decreased leakage power consumption. Alternatively, proposed cell with sleep transistors and cell with dual threshold voltage (DVT) is designed, also.

The evaluated parameters of all the circuits are compared with conventional 6T SRAM cell. When comparison of the different parameters carried out, the new 8T SRAM cell shows the data stability is increased up to 47.5% and 33.72% during read operation and hold state respectively as compared to conventional 6T SRAM cell. The results depict, read and write access delay is deducted up to 27.9x and 55.6x respectively when compared with 6T SRAM cell. Additionally, the new 8T SRAM circuit with DVT consumes 2.842pA leakage current, which is 92.05% less than 6t SRAM cell.

8T SRAM cell design to enhance the stability and reduce the leakage power. For validation of proposed 8T SRAM cell, compared results with reported data. The parameters used in proposed cell are comparable to the existing 8T SRAM cell at same technology and design rules. The stability of proposed cell has been analysed using N-curve metrics.

2. SRAM

2.1 INTRODUCTION

The portable devices such as hand held mobile devices and personal digital assistants are gaining more popularity as well as making changes in every aspect of our daily lives. The major concern enhancing the demand for portable device market is multimedia data processing, which includes the image/video applications [1]. Video applications require a large amount of embedded memory access, which results in significant power consumption and thus limits the battery life time. Power dissipation has become an important consideration due to the increased integration and operating speeds, as well as due to the explosive growth of battery operated appliances [2]. Low power design is a, buzzword these days and designing with low power requirements has

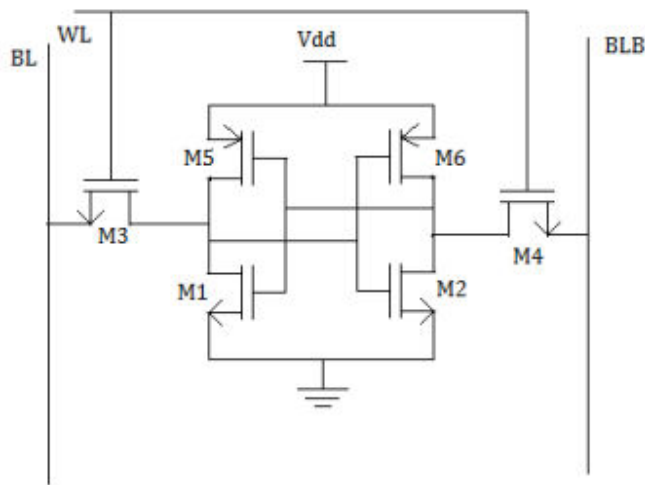
been always an important aspect of video applications. The overarching reason why the low power design is becoming so important today is the increase of leakage current with the shrinkage of device dimension. Low power design is indispensable to realize battery operated systems. Due to the limited size of handheld devices, it is impossible to use larger batteries in it and short battery life time of a smaller battery limits the use of them [3]. Therefore low power design is essential to extend the battery life time. Most of the digital devices consists of memories and hence reducing power consumption of memories as well as area reduction is paramount important as of today to improve system performance, efficiency and reliability. One of the effective ways to reduce the dynamic power consumption is lowering the operating voltage due to its quadratic relationship [4]. In scaled technologies maintaining high SRAM yield becomes more challenging since they are particularly vulnerable to process variations due to the minimum sized devices used in SRAM bit cells.

The amount of embedded SRAM in modern Systems on-Chips (SoC) increases to meet the performance requirements in each technology generation [6]. As the International Technology Roadmap for Semiconductors (ITRS) predicts, memory area is becoming 90% of the total chip area [7]. This trend is continuing also for real time video System on Chip (SoC). Two aspects are important for SRAM cell design: (i) the cell area and (ii) stability of cell. In today's technology power dissipation in the memory circuits has become an important design consideration. As technology scaling, more devices are integrated into the system, as a result the corresponding leakage power increases. Lower voltages and smaller device dimensions have a significant effect on data stability, dynamic power consumption and leakage power in SRAM cell [8]. The rest of this paper is organized as follows: section 2 briefly presents the bulk CMOS 6T SRAM cell and its failures. Section 3 presents the 8T SRAM cell reported in [9]. In section 4, describes the proposed 8T SRAM cell and analyze its effectiveness. Section 5 discusses the stability metrics. Section 6 discusses the experimental conclusion between the cell presented in [9] & proposed cell, and finally some conclusions are drawn in section 7. In this work 65nm technology used for the simulation of all results.

2.2 6T SRAM

As shown in figure 1, 6T SRAM cell design uses bi-stable latching circuitry to store a bit (M1, M2, M5 & M6) and two access transistors (M3 & M4). Word Line (WL) is connected to the

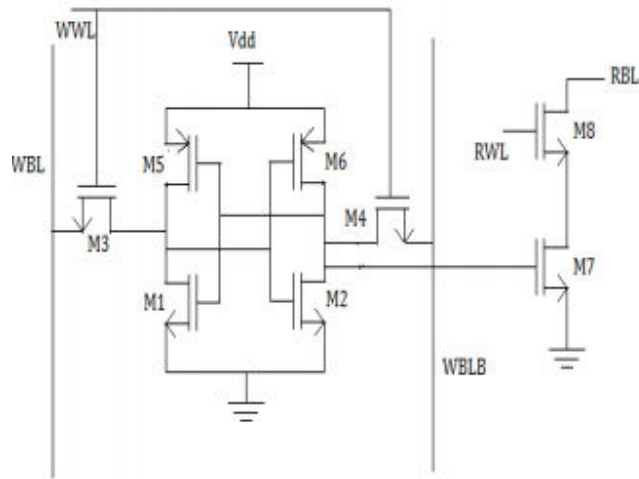
access transistors at their respective gate terminals. WL is used to select the cell. Source/Drain terminals are connected to the Bit Lines (BL & BLB), which are used to perform the read and write operations on the cell [5]. The problem associated with bulk MOSFET based 6T SRAM cell during read operation is, when the WL is turned ON, it raises the output voltage at node that stores „0“, which could turn ON the opposite inverter pull down transistor, when this happens the voltage at node which stores „1“ will be reduced. This voltage may drop little, but it should not drop below the threshold voltage. If it drops below the threshold voltage of MOSFET, it leads to read destructive operation. Due to this stability of the 6T SRAM cell will be degraded. The operation of writing is accomplished by forcing one bit line low while other bit line remains at about $V_{DD}/2$, which could leads to enhancement in dynamic power consumption. With technology scaling, in the new coming manufacturing process the operating voltage and threshold voltage decrease and it demolishes the stability of the SRAM cell. Due to the direct paths between bit lines to the storage nodes, the data stored in conventional SRAM cell easily deteriorated by the external noise. Based on the above reasons bulk MOSFET based 6T SRAM cell is not suitable for real time video applications [10]. Hence we require a new design for high stability, low dynamic and leakage power.



2.3 8T SRAM

To address the read destructive problem, the read and write operations are separated by adding transistor stack to the conventional 6T SRAM cell, thus it has the area penalty but operates

efficiently than the 6T SRAM cell at lower . The 8T SRAM circuit described in this section [9]. The schematic of the 8T SRAM cell with transistors sized for a 65-nm CMOS technology.



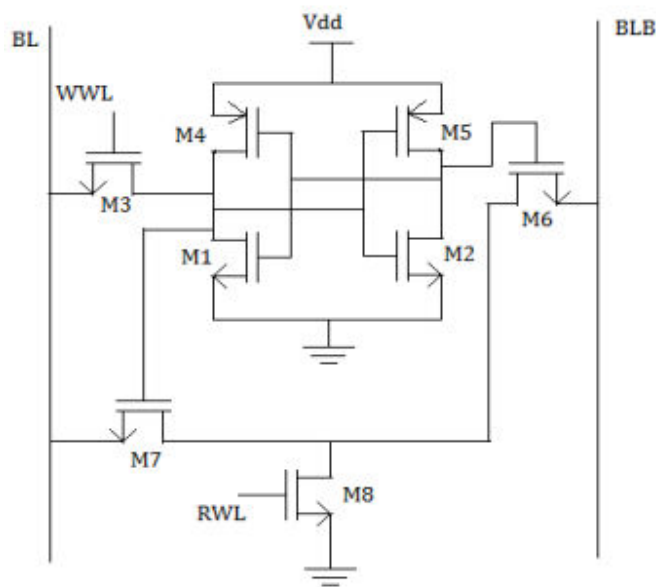
The disturbance of bit lines during read operation is the primary source of instability problem in SRAM operation. The stability in 8T SRAM cell can be enhanced by isolating the read port from the write bit lines. The 8T SRAM cell composed of conventional 6T SRAM cell for writing operation and a transistor stack, which can be used for read operation. The read and write operations are controlled by separate signals Write Word Line (WWL) and Read Word Line (RWL). During the read operation Read Bit Line (RBL) is pre charged to and WWL is maintained at . Depends on the value stored in cross coupled inverters RBL, discharges (or) maintained at . If RBL discharges, it can be treated as the stored bit is „1“, otherwise it is „0“. The storage nodes are completely isolated from the write bit lines, which can increase the stability of the SRAM cell. During the write operation WBL and WBLB lines are pre charged to predetermined values. Then, asserting the write word line WWL and nodes attain the corresponding values from the bit lines. It uses the two additional Word Lines to perform read and write operations, when compared with 6T SRAM cell, which could increase the metal density, wire delay and dynamic power consumption and leakage power.

3. PROPOSED DESIGN FOR 8T SRAM CIRCUITS

A novel 8T SRAM cell structure to reduce the leakage current and dynamic power consumption has been reported in this work. The schematic of proposed 8T SRAM cell at 65nm technology is as shown in fig. 3. The proposed SRAM cell composed of write access transistor (M3),

controlled by Write Word Line (WWL) and read access transistor (M8) is controlled by the Read Word Line (RWL). During the write operation WWL is transitions to high value and RWL and BLB both are maintained at Hence, the read access transistor (M8) cut OFF. To write „1“ into the cell Bit Line (BL) is pre charged to a high value, then „1“ is forced through the write access transistor (M3). Similarly, to write „0“ into the cell, BL is discharged. Hence, to perform write operation the proposed cell utilizing single BL, which could leads to reduction in the dynamic power consumption and leakage power.

During read operation, RWL is transition to high value and WWL is maintained at Hence the write access transistor is cut OFF. Prior to read operation BL and BLB are pre charged to . Assume that „1“ is stored left and „0“ is stored right side, then BL discharged through M7 and M8. Since, „M6“ is cut OFF there is no path to discharge the BLB. Hence BLB is held at high value. Alternatively, if „1“ is stored right side, BLB is discharged through „M6“ and „M8“. Since, „M7“ is cut OFF there is no path exists to discharge the BL. Hence, it can maintain at high value. With this, storage nodes completely isolated from the Bit Lines (BL) during read operation, hence stability increases significantly.

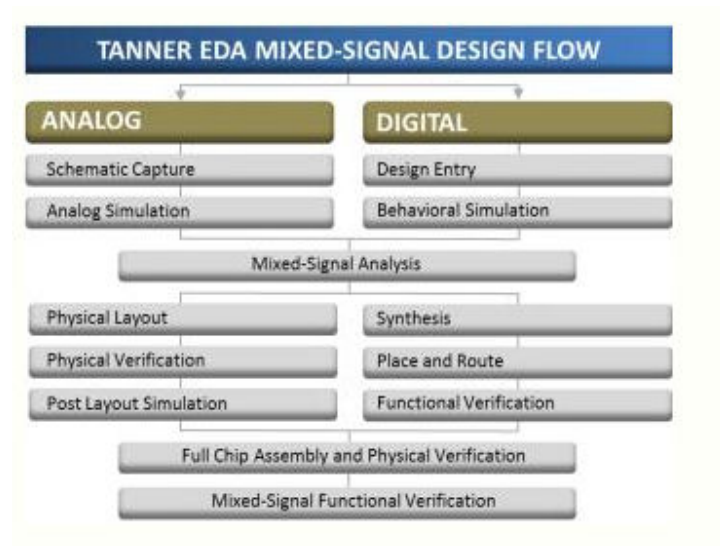


5. TANNER TOOLS

5.1 INTRODUCTION

Tanner EDA by Mentor Graphics now enjoys the stability that comes with joining a successful big 3 EDA company. Mentor will continue to invest in the Tanner EDA suite to strengthen it as the user base and support Tanner EDA products, their customers and their foundry partners and place particular focus on IoT users who design and integrate: power electronics, sensors, MEMS, photonics and RF circuitry.

Tanner EDA solutions by Mentor Graphics provides a complete line of EDA software tools that drive innovation for the design, layout, and verification to tape-out of analog and digital integrated circuits (ICs). Tanner EDA solutions offer a complete design environment supporting analog, mixed-signal, or MEMS domains in one highly-integrated end-to-end flow. Our customers are creating breakthrough applications in areas such as Automotive, Life Sciences, imaging, sensors, power management, Military, Aerospace, Space, Internet of Things (IoT) and ARM Powered® solutions. Tanner EDA products have a low learning curve, high interoperability, and a powerful user interface improve design team productivity.



3.2 WORKING

Tanner EDA is a suite of tools for the design of integrated circuits. These tools allow you to enter schematics, perform SPICE simulations, do physical design (i.e., chip layout), and perform design rule checks (DRC) and layout versus schematic (LVS) checks. There are 3 tools that are used for this process: S-edit - a schematic capture tool T-SPICE - the SPICE simulation engine integrated with S-edit L-edit - the physical design tool

3.2.1 Using S-Edit (Schematic Entry Tool) & T-SPICE (Analog Simulation Tool)

S-edit is a schematic entry tool that is used to document circuits that can be driven forward into a layout of an integrated circuit. It also provides the ability to perform SPICE simulations of the circuits using a simulation engine called T-SPICE. T-SPICE can be setup and invoked from within S-edit.

Part 1: Setup your Directory Structure & download Libraries

- a) Log onto a computer on 6th floor Cobleigh.
- b) You want to create a directory for all of your Tanner EDA projects. You also will need to download and unzip a set of library & model files from the course website that will be used for your simulations. - Create a directory structure named "EELE414_VLSI_Fall2011\Tanner_Projects"
- c) Go to the course website and download the zip file called "Tanner_Libraries.zip". Unzip it into your Tanner Projects directory. This group of files contain the necessary information to enter components into S-edit (circuit symbols), perform SPICE simulations (models), and do physical layout (layer definitions, DRC, LVS)

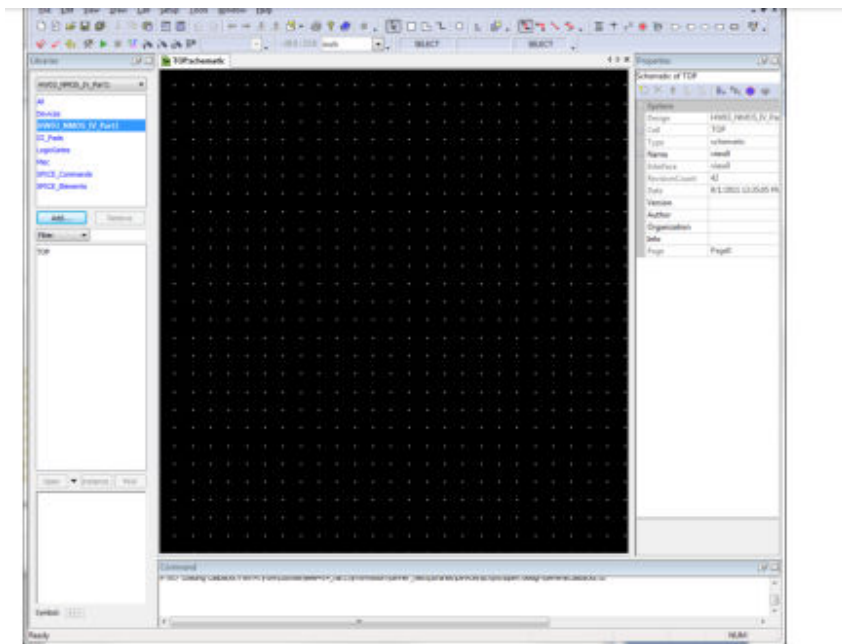
Part 2: Start a New Design & Setup Libraries a) Start S-Edit: - Start – All Programs – Tanner EDA – Tanner Tools v12.6 – S-Edit v12.6 b) Start a New Design: Using the pull down menus, create a new design: - File – New - New Design A dialog will appear asking for a design name and location. When you give the name, Sedit will create a folder of that name in the directory that you provide that will contain all of the design files. You should give a descriptive name that represents each simulation you will be running.

- Enter the name "HW03_NMOS_IV_Part1" and browse to your "EELE414_VLSI_Fall2011\Tanner_Projects" directory - Click "OK" c) Create a new Cell A "cell" is a design element. A cell can contain multiple views such as schematics and 3 symbols. Cells can be instantiated in other cells. When performing a simulation, we will typically call the cell "TOP". When we are testing a circuit, for example an inverter, the inverter will have its own cell that contains a schematic of the devices and a symbol. The inverter cell is instantiated in the TOP cell that contains ideal elements such as voltage sources and probes that are only used for simulation.

This allows us to separate the cells that are actually going to be implemented on the die versus cells that are only used for simulation. Using the pull down menus, create a new cell view:

- Cell – New View: - enter the cell name "TOP". Ensure the design name is "HW03_NMOS_IV_Part1" and click OK. You can leave the interface and view names "view0". A blank schematic page will appear. It is a good idea to save this right now.

d) Enter the symbol libraries: First, you need to include a library which contains the symbols for all basic circuit elements such as resistors, NMOS, capacitors, etc... The libraries for all the basic symbols are in the Tanner_Libraries.zip file you downloaded and unzipped. - On the left side of the S-edit screen you'll see a Libraries window, click on the "Add" button. - Browse to "Libraries\All\All.tanner" and click "OK" You should see a set of libraries appear:



e) Setup the SPICE Models for the Generic_025 kit. The libraries that you just added have symbols for NMOS and PMOS transistors. However, all non-linear components such as MOS transistors require a model to describe their behavior.

If you simply enter an NMOS symbol in your schematic, SPICE will not know what to do since each NMOS transistor fabricated in a different technology behave differently. In this example, we will use a transistor technology called “Generic_025”, which represents a standard, 0.25um CMOS process.

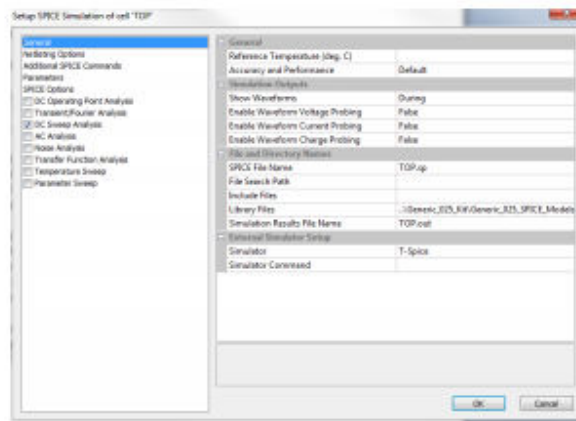
You will need to setup the SPICE models for this process in S-edit. Once you do that, when you enter an NMOS or PMOS transistor, you can then associate the 0.25um model to that symbol. Using the pull down menus, setup the SPICE models: - Setup – SPICE Simulation - In the dialog that appears, you should highlight “General” on the left. - On the right, click in the “Library Files” field. This is where you will specify any SPICE models you will be using in your simulations.

Browse and select “Generic_025_Kit\Generic_025_SPICE_Models_Level1.lib”

- On the right, click in the “SPICE File Name” field. This is where you specify the name and location of the SPICE Netlist output. Browse to your design directory “EELE414_VLSI_Fall2011\Tanner_Projects\HW03_NMOS_IV_Part1” and enter the filename “TOP.sp”.

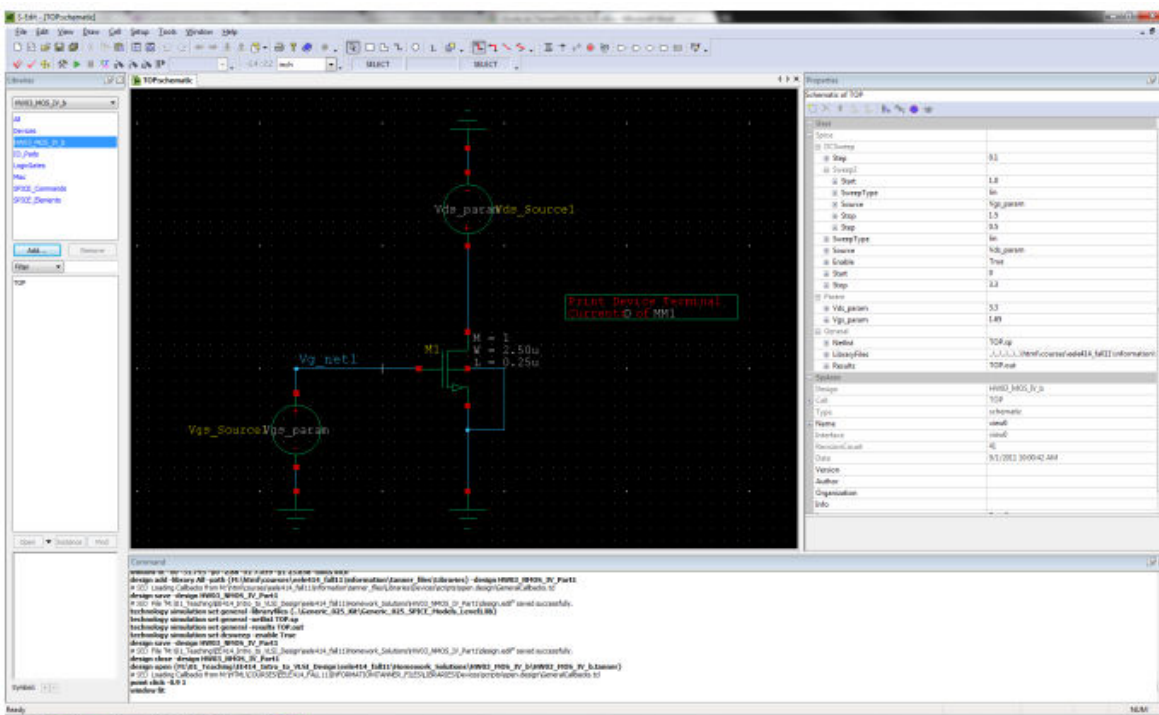
- On the right, click in the “Simulations Results File Name” field. This is where the results of the simulation will be written. This file is what the waveform viewer will look for when you go to plot your results. Browse to your design directory

“EEL414_VLSI_Fall2011\Tanner_Projects\HW03_NMOS_IV_Part1” and enter the filename “TOP.out”. - Before you can exit this window, you will need to select an analysis type. We will setup the details of the analysis later, but for now, just check the “DC Sweep Analysis” and click



“OK” to close the setup window.

Part 3: Enter the Schematic to simulate the IV behavior of an NMOS Transistor We will be entering the following circuit.

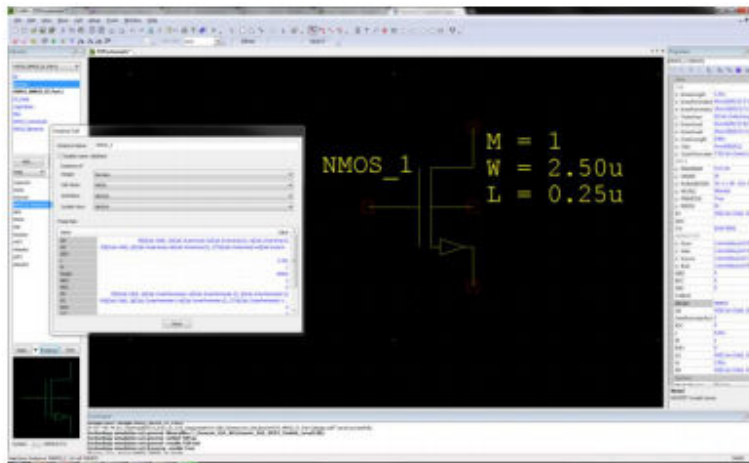


a) Enter the NMOS transistor

- On the left, click on “Devices” in the upper window. This will display all of the symbols available in this group. You should see all of the components that you can implement on a CMOS integrated circuit.

- On the bottom left window, click once on “NMOS”. You should see the symbol of the NMOS transistor show up in the symbol viewer window at the bottom.

- To place the NMOS, you will click on the “Instance” button. Two things happen when you click on this button. First, a dialog will appear that will allow you to setup the parameters for the NMOS. Second, the symbol will attach to your mouse. We will place the NMOS in the schematic first and then set its properties later. This is an easier way to enter the device. Click in the schematic window to drop an instance of the NMOS. Hit the “Esc” button to end the insert-mode.



The NMOS is now in the schematic. A note on zooming: - [Home] = zoom fit - [-] = zoom out - [=] = zoom in - the scroll wheel also zooms in/out.

- To setup the NMOS, click on the NMOS symbol. You will see the properties of the device on the left. We want to setup the following:

- Model : enter “NMOS”. This model is found in the Generic_025 library you added - Name: M1. The SPICE designation for MOS transistors is to have the name start with an “M”. S-edit automatically appends an M to the name is the final name will be “MM1” in the TOP.sp file.

But it is good practice to name all MOS transistors with M’s. - W Set to 2.5u. This is the default. - L Set to 0.25u. This is the default. b) Enter a DC source for VGS

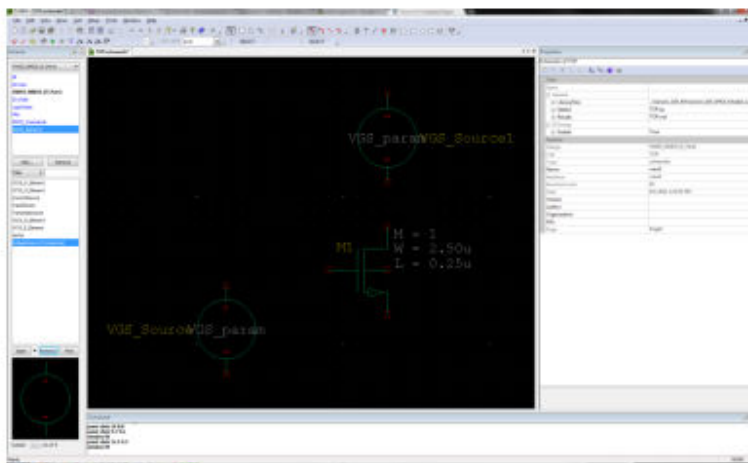
- Using the same process you used for the NMOS symbol, enter a “SPICE_Elements:VoltageSource”. This is a generic voltage source symbol that is configured as a DC, TRAN, PWL, etc.. in its properties dialog.

- Click on the voltage source and enter the following: - MasterInterface: DC (this is the default but this is how you would change it to something else.

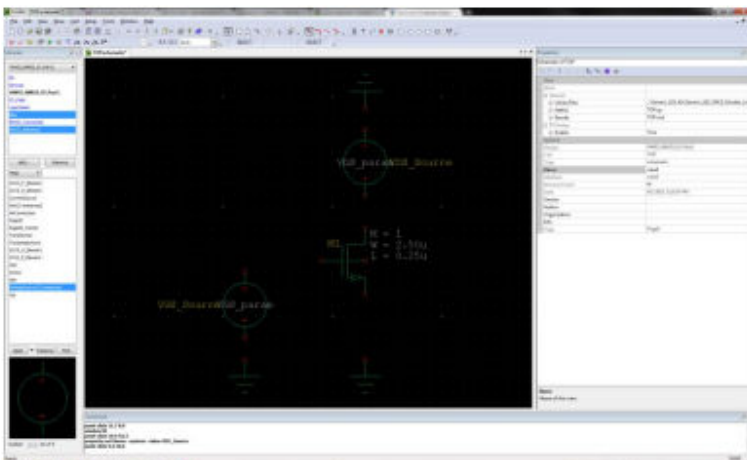
- Name: VGS_Source (it is a good idea to use descriptive names) 7 - V This is where you will set the DC voltage (i.e., 4v, 5v). However, for this example we will use a parameter instead of a hardcoded value. We will enter a parameter name here and then set up the parameter later. Enter “VGS_param” for the value of V. When performing a DC sweep, you must use parameters for the sweep.

c) Enter a DC source for VDS - Using the same process as above, enter a DC source for VGS with the following:

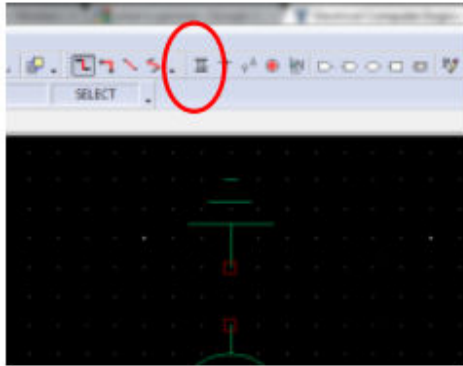
- MasterInterface: DC (this is the default but this is how you would change it to something else.
- Name: VDS_Source - V “VDS_param” Position the sources as in the following figure:



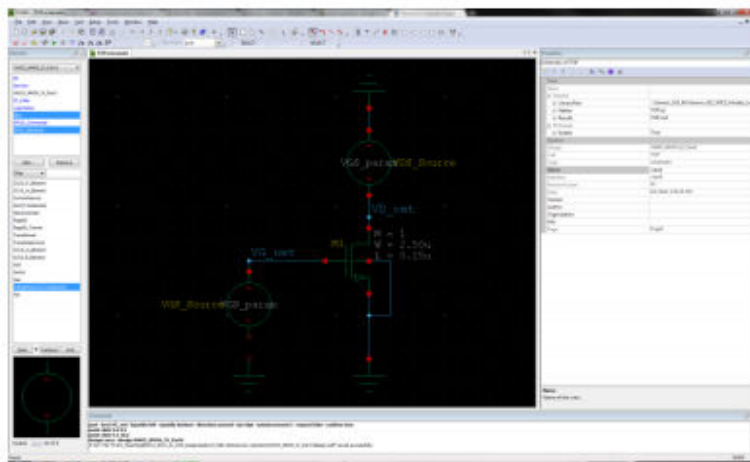
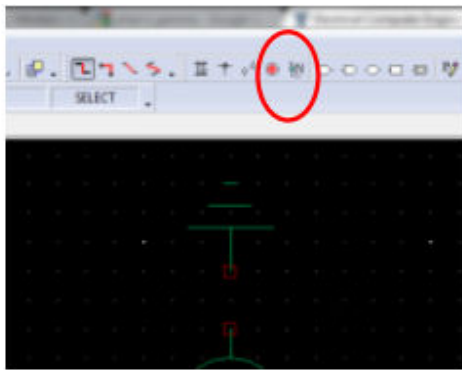
A note on zooming: - Hold down ALT-M to move a component. While holding these buttons down, click and drag the components. - To rotate, click on the device and click the [r] button. 8 d) Enter Grounds - Using the same process as above, enter 3 grounds from Misc:Gnd



e) Enter Wires - You can enter wires by clicking on the “wire” icon at the top



- Enter wires by clicking on a symbol node and then dragging. Enter corners by clicking once where you want to turn.
- You can label nets using the “Net Label” icon at the top



f) Enter a Current Probe to monitor IDS - Enter the SPICE_Commands:PrintCurrent component. This doesn't connect to anything. You just place it anywhere and then tell it what current to monitor in its properties dialog.

- In its properties dialog, setup: Terminal: D (this is the Drain of the NMOS) Device: MM1 (this is the name of the device. Notice that we called it M1, but S-edit automatically appends another M to the name. You will only see this once you run the Netlist. Analysis: DC (VERY IMPORTANT TO SELECT THIS!!!!)

Part 4: Setup the Parameters that will be used during the DC sweep analysis When we entered the VGS and VDS sources, we set their values to "VGS_param" and "VDS_param". We now need to setup these parameters. Using the pull down menus: - Setup

- SPICE Simulations - On the left, click on "Parameters"

- On the right, click on the "Add Parameters" button (it is in the upper right corner next to the red

X) Enter: Name: VGS_param Value: 1v - On the right, click on the "Add Parameters" button Enter: Name: VDS_param Value: 2.5v 10 We will overwrite these values during our sweep, but the parameters need to exist first.

Part 5: Setup the SPICE DC Sweep Analysis Using the pull down menus: - Setup

- SPICE Simulations - On the left, click on "DC Sweep Analysis"

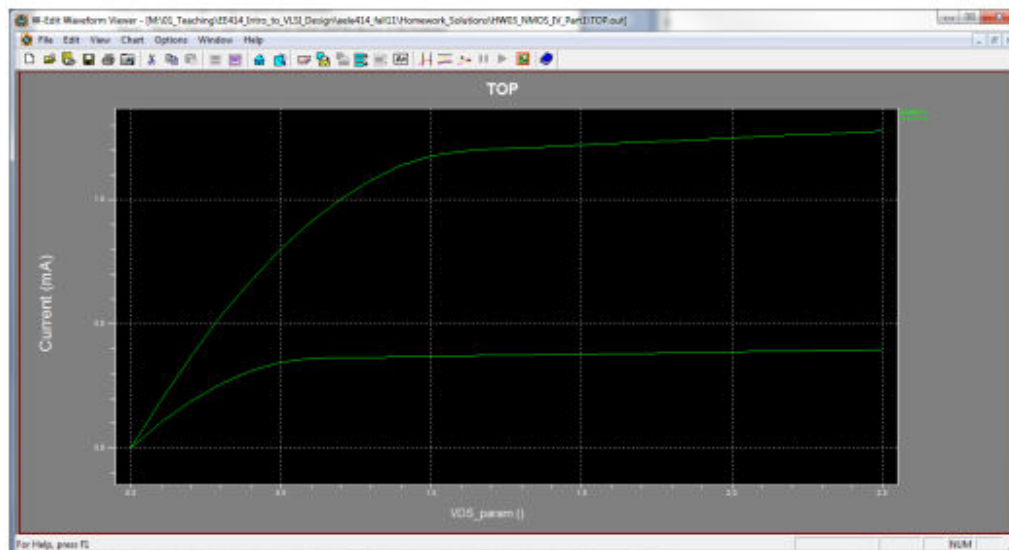
- On the right, enter the following for Source (this is what will be swept) Source or Parameter Name: VDS_param Start Value: 0 Stop Value: 2.5 Step: 0.1 Sweep Type: lin - On the right, enter the following for Source (this is what will be swept)

Source or Parameter Name: VGS_param Start Value: 1 Stop Value: 1.5 Step: 0.5 Sweep Type: lin

NOTE: The first parameter you setup in this dialog will be plotted on the independent axis. Part 6: Simulate the Design a) First, check you design using the pull down menus:

- Tools – Design Checks (any warnings or errors will be shown at the bottom) b) Simulate your design: - Click on the Green Arrow to start the simulator:

The T-Spice window will appear. If everything is OK, the waveform viewer will also appear. If everything worked, your waveforms should look like this:



c) View the Netlist: - In the T-spice window, right click on the file at the bottom and select “Show Netlist” This will bring up the TOP.sp Netlist that was created and used by the spice engine. This is a good place to look when you get errors. This is the text based description of what you entered in S-edit.

```

T-Spice - [TOP.sp]
File Edit View Simulation Table Setup Window Help
* SPICE export to: ASCII 12.61
* Export time: Wed Sep 07 15:22:56 2011
* Design: M03_1805_1V_Part1
* Cell: TOP
* View: view0
* Export net: top-level netlist
* Export model: hierarchical
* Exclude .model: no
* Exclude .end: no
* Expand paths: yes
* Wrap lines: no
* Root path: M:\01_Teaching\EE414_Intro_to_VLSI_Design\ee414_fall11\Tanner_Projects\M03_1805_1V_Part1
* Exclude global paths: no
* Control property name: SPICE

***** Simulation Settings - General section *****
.lib M:\01_Teaching\EE414_Intro_to_VLSI_Design\ee414_fall11\Tanner_Projects\Generic_GDS_Kic\Generic_GDS_SPICE_Models_Level1.lib*

***** Simulation Settings - Parameters and SPICE Options *****
.param VDS_param = 1
.param VDS_param = 2.5

***** Devices: SPICE.CHEN > 0 *****
M01 Vd_net Vd_net Gnd Gnd MOS n=2.5u l=250u AS=2.25p FS=6.5u AD=2.25p RC=6.5u
VDS_source Vd_net Gnd DC VDS_param
.PRINT DC IS (M01)

***** Simulation Settings - Analysis section *****
.AC lin VDS_param 0 2.5 0.1 lin VDS_param 1 1.5 0.5

***** Simulation Settings - Additional SPICE commands *****
.end

Status Input File Output File Options Start/End Time Elapsed
Loaded M03_1805_1V_Part1 TOP.sp Top.sp September 07, 2011 15:22:26 00:0
Ready
Ln 25, Col 37 NUM

```

d) View the Waveform: - If the windows viewer did NOT automatically appear, you can click on the file in the Tspice window and select “Show Waveform”.

3.3 Transient Analysis of a CMOS Inverter & Symbol Creation

Part 1: Start a New Design, Setup Libraries & Setup Simulation a) In this example, we will create a CMOS inverter and simulate its transient response. We will create an inverter design that contains a symbol and then instantiate it in another schematic to stimulate the circuit. Symbols are handled by adding another view to a design.

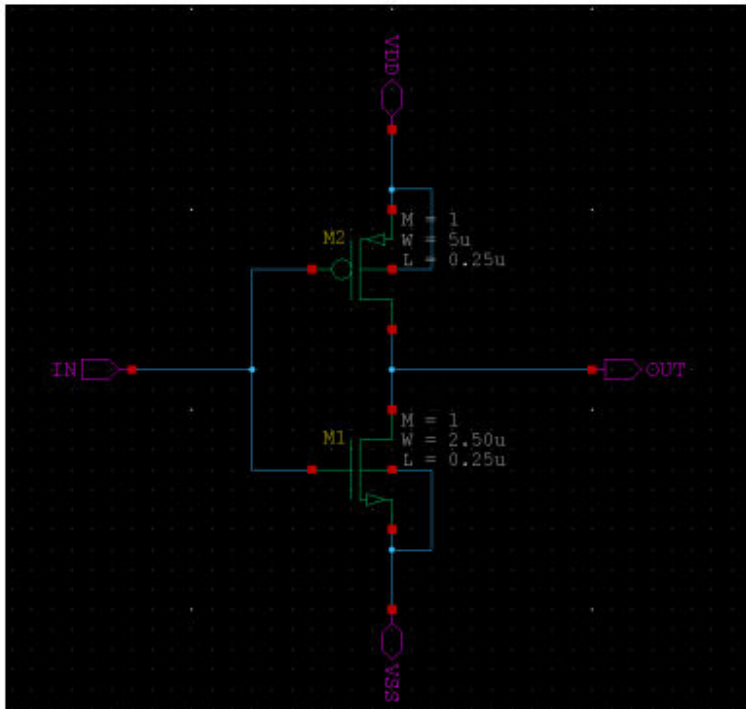
We will start by creating a design called “Inverter” and then create a schematic view. This schematic will contain an NMOS and PMOS wired as an inverter. We will add “Ports” for the Input, Output, VDD, and VSS. We will then add a symbol view to this design. The symbol will contain the inverter shape and the corresponding pins for Input, Output, VDD, and VSS.

We will then create a separate schematic called TOP that will be used to test the inverter. We will instantiate the inverter symbol in TOP. We only want to put items into the inverter design that can be fabricated. TOP will contain the ideal voltage sources to provide the input waveform, the power supplies, and a mock load.

In this way, when we go into physical design (i.e., layout), we only drive forward the isolated circuits.

- Start S-Edit - Create a new design called: “EELE414_VLSI_Fall2011\Tanner_Projects\HW04_INV_Transient_Part1” - Add the Tanner_Projects\Libraries\All\All.tanner library to the library list on the left

- Create a new Cell called “TOP” using the Pull Down Menus - Cell - New View Name = TOP View Type = schematic - Setup the simulation using the Pull Down Menus: - Setup – SPICE Simulation - Highlight the General Tab of the Setup SPICE window and set the following: SPICE File Name: \HW04_INV_Transient_Part1\TOP.sp Library Files: ..\Generic_025_Kit\Generic_025_SPICE_Models_Level1.lib Simulation Results File Name: File Name: \HW04_INV_Transient_Part1\TOP.out 13 - Check the “Transient/Fourier Analysis” box on the left and set the following: Stop Time = 2ns Maximum Time Step = 10ps - Click “OK” Part 2: Create the Inverter a) Create a new schematic view using the pull-down menus: - Cell - New View Name = Inverter View Type = schematic b) Enter the inverter schematic:



- Entering the NMOS: Name = M1 L = 0.25u W = 2.5u Model =NMOS 14

- Entering the PMOS: Name = M2 L = 0.25u W = 5.0u Model =PMOS - Entering the Ports: Ports are entered using the icons on the top of the S-edit window.

Enter the following: In Port: Name it “IN” Out Port: Name it “OUT” In/Out Port: Name it “VDD” In/Out Port: Name it “VSS”

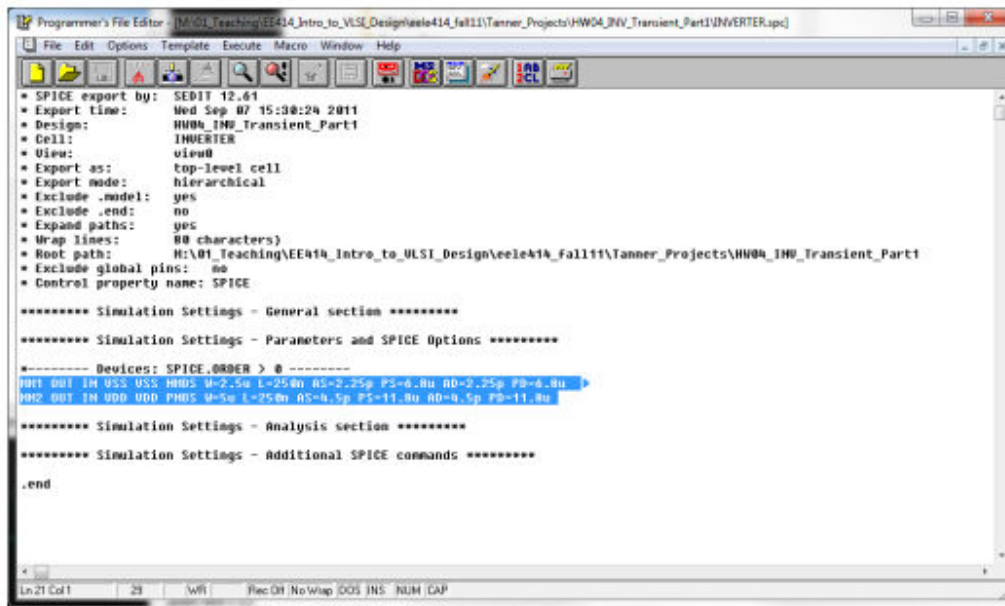
- Wire up the Inverter Enter wire connections as shown in the previous figure.

c) Export a SPICE Netlist Exporting a SPICE Netlist is a good idea in order to verify that you have entered the schematic correctly. Also, this Netlist will be used later when performing a “Layout versus Schematic (LVS)” check.

We want to export a Netlist at the Inverter schematic cell level so that a Netlist of just the inverter exists for LVS. When we conduct the simulation of this inverter, we will create a TOP level schematic that will have a Netlist containing ideal voltage sources. This Netlist can’t be used for LVS since it contains components that won’t be fabricated. With the schematic open, use the pull down menus to perform:

- File – Export – Export SPICE. - Browse to your design directory and give the file name “Inverter.spc”.

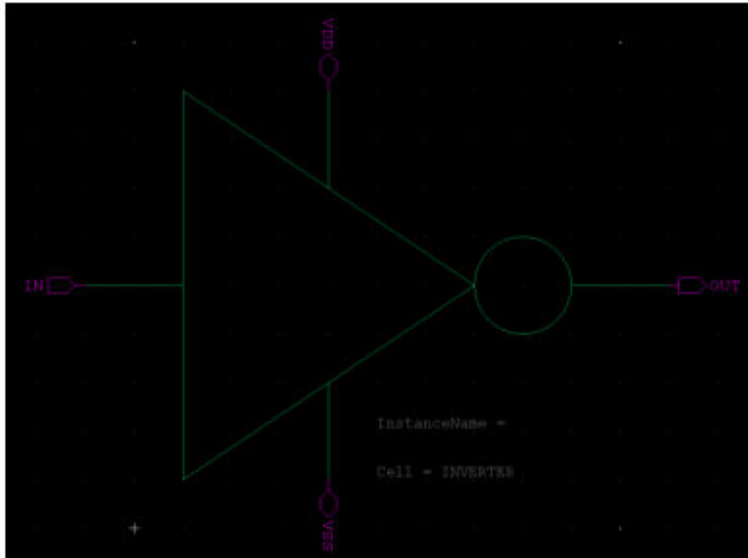
- Click “OK” If you open the Inverter.spc with a text editor, you will see the following:



d) Create the Inverter Symbol Symbols can either be created manually by creating a new symbol view or automatically by S-edit. We will use the automatic symbol generation. This will create a new symbol view from the schematic, create the ports for the symbol, and make a symbol shape. While the shape of the symbol is rarely what we ultimately want, it will do a lot of the work for us.

- With the schematic view open, use the pull down menus to create the symbol view:

Cell – Update Symbol A new window will come up with a square symbol and 4 ports with the same names you entered in the Inverter:schematic view (i.e., IN, OUT, VDD, VSS). You should edit the shapes until you have created a symbol that looks like an inverter:



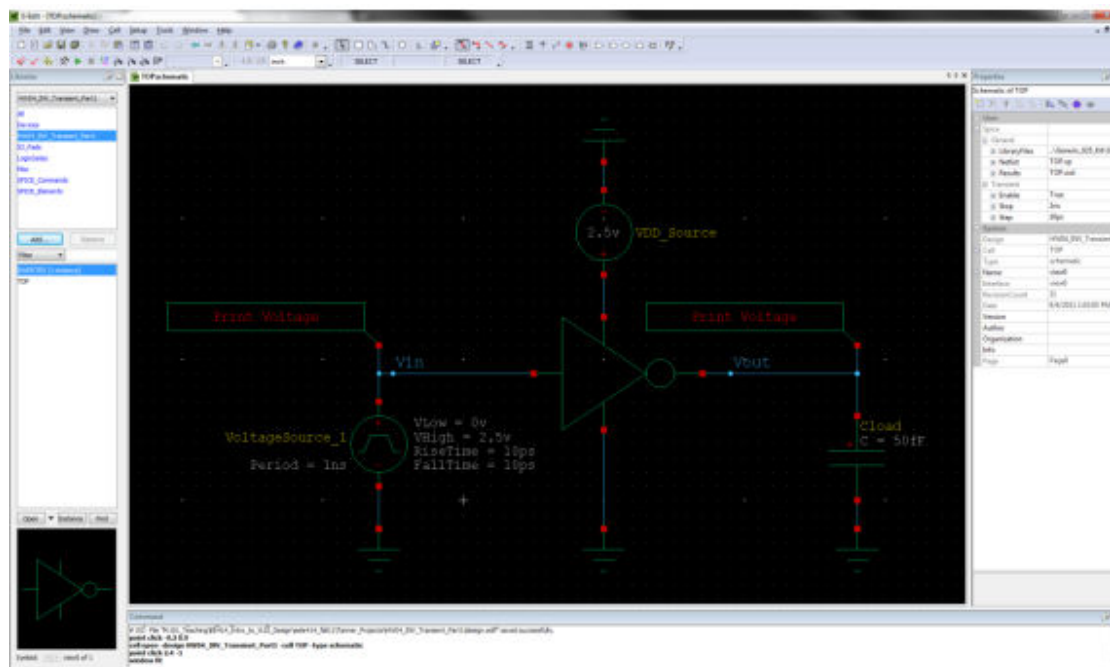
A note on drawing: The “Path” icon will put you into a mode where you can draw lines that are not wires. The “Circle” icon will allow you to enter the inversion bubble. The ports can be moved by holding down “alt-m” The ports can be rotated by selecting and pressing the “r” button Remember to save.

Part 3: Create the TOP schematic to test the Inverter

a) Instantiate the Inverter in the TOP schematic Open the TOP schematic view using the pull-down menus:

- Cell – Open View: Cell Name: TOP View Type: schematic In the library windows on the left of the window, highlight your “HW04_INV_Transient_Part1” library. In the lower left window, you will see your two Cells “TOP” and “Inverter”.

- Click on “Inverter” and you will see your symbol show up in the symbol viewer. - Click on the “Instance” button and place your symbol in the TOP schematic. 17 b) Enter the following circuit in order to power and stimulate your inverter:



- Enter the Pulse Voltage Source.

All voltage sources are the same component in the SPICE_Elements library. The default is DC, but this can be changed to any other type of source in the properties dialog.

Name = Vin_Source MasterInterface = Pulse Period = 1ns PulseWidth = 0.5ns VHigh = 2.5v VLow = 0v RiseTime = 10ps FallTime = 10ps - Enter a Load capacitor from the Devices library. Name = Cload C = 50fF - Enter a DC Source for VDD Name = VDD_Source MasterInterface = DC V = 2.5v

- Enter the grounds from the Misc library 18

- Enter wire connections and name them Vin and Vout

- Enter a voltage probe for both Vin and Vout Part 4: Simulate the Design

a) First, check you design using the pull down menus: - Tools – Design Checks (any warnings or errors will be shown at the bottom)

b) Simulate your design: - Click on the Green Arrow to start the simulator: The T-Spice window will appear. If everything is OK, the waveform viewer will also appear. If everything worked, your waveforms should look like this:

